

L6 总结

L6 总结:上下文管理 + Prompt/Context Engineering

JD 权重:上下文管理 4/10 + Prompt Engineering 5/10。也是学习者从 L2 就惦记的问题 (“messages 越滚越长”)。本节还补 L1-L5 缺口 2(结构化输出)。 本节最大的价值不在讲义,在作业里用手撞出来的一连串真实道理。

一、上下文管理

1. 为什么必然会爆:平方级的账

API 无状态 → 每轮重发全部历史 → 累计 = $1+2+\dots+N \approx N^2/2$ 。token 是平方级增长,不是线性。学习者在作业里亲眼看到 累计输入 涨得远比直觉快。

2. 四个后果(第 4 个最反直觉)

超限 / 变贵 / 变慢 / 质量下降(“lost in the middle” :长上下文里中间信息被忽略)。 > 核心:窗口大 ≠ 该塞满。

3. 上下文是 “预算表”

system / 工具 schema / 对话历史 / 工具返回 / RAG 结果 / 用户输入,都在抢有限预算。 Context Engineering = 每轮决定放什么、放多少、什么形式。

4. 四类策略

截断 / 摘要 / 混合(最常用:最近 N 条原文 + 更早的摘要) / 外置(= L5 记忆 + L7 RAG)。 触发方式:阈值触发的惰性优化(平时不管,超阈值才压一次)。

二、Prompt Engineering(从学习者 L5/L6 翻车提炼)

- 五块骨架:角色 / 任务 / 约束(必须做+绝不能做,最值钱) / 示例 / 输出格式
- 别用模糊限定词(“尽量/必要时” = 给模型留偷懒空间);L5 的 remember 就栽在这
- 点名禁掉已知失败模式(“绝对不要只嘴上说已记住却不调工具”)
- prompt 是迭代出来的;改前先看模型实际干了什么;prompt 每轮重发,也占预算 → 简洁而精确

三、结构化输出(补缺口 2)

三层次:①prompt 请求(无保证)②JSON mode(保证合法 JSON)③schema 强制(字段/类型/必填全保证)。洞察:**Function Calling 本质就是结构化输出**(给 schema,返回符合 schema 的 arguments)。选型:要触发动作→function calling;只是数据给代码消费→结构化输出;给人看→普通文本。 > (动手部分 Part B 结构化路由并入阶段二 capstone。)

四、作业:从“能跑的脚本”升级为“生产级可测模块”

用户主动要求:后续课程一律对齐生产规范,玩具写法会在面试穿帮。于是 L6 作业重做成 homework/v2/ 三文件分层: - context.py :纯逻辑(不碰 LLM/网络), find_safe_cut_index (纯函数)+ ContextManager - test_context.py :单元测试(教练代写,用户选择先专注实现、测试留待后续系统学) - agent.py :接线层(碰 LLM/网络)

三条生产规范贯穿:①纯逻辑与副作用分离(依赖注入摘要器 → 测试快/稳/免费/能测异常) ②不修改入参 ③返回结果对象(CompactResult)而非裸值。对照面试:蔚蓝 JD “建立评估与回归测试体系”。

五、实战撞出来的道理(本节精华,全部进 interview-notes)

1. token 估算别用玩具法

`len(content)//2` 幼稚且**系统性漏算工具 schema**(每轮几百 token)。生产做法:用 API 返回的 `response.usage.prompt_tokens` (权威、含 schema、零依赖)。真要发请求前精确算,才引入 tiktoken/HF tokenizer(注意 DeepSeek/Qwen 分词器不同,tiktoken 对不上)。

2. 假绿:测试通过 ≠ 验证了什么

一批 pass 空壳测试全部“通过”。**没有断言的测试永远绿**。识破:看有没有真断言;防住:TDD 先让它红(“没见过红的测试不可信”)、变异测试。同一主线第 N 次:成功信号 ≠ 达成目标(L5 tool_calls、L6 假绿)。

3. 混血列表的根治

`messages` 混了 dict(手写)和 SDK 对象(append)→ `AttributeError`(第 3 次犯)。根治:在入口

`msg.model_dump(exclude_none=True)` 归一化,下游全免疫。金句:同一个 bug 在多处重复出现,说明抽象层漏了——在数据入口统一格式,而非每个使用点做防御。

4. 切点安全:危险的是 tool 不是 assistant

压缩切点若落在 tool 消息上 → 父 assistant 被切走 → 孤儿 tool → API 报错。用 `while` 一路退过连续 tool(并行工具调用会有多条),退到父 assistant。

5. 阈值必须高于“压缩地板价”

阈值 $600 <$ 压缩后的固定开销(~ 750) → 压完还超 → 每轮都压(thrashing)。

6. 反复摘要 = 复印件的复印件

thrashing 导致每轮把上一轮的摘要再压一次 → 名字“小明”在第 2 轮被压没,模型还写“用户身份信息无”。重复有损压缩必然衰减信息。阈值调到 1200(高于地板) → 单次压缩 → 名字保住。

7. 模型可用性以端点为准

我建议换 `deepseek-chat`,但该端点只有 `deepseek-v4-pro` / `deepseek-v4-flash` (400 报错)。基础设施事实(模型名/额度/限流)以当前环境返回为准,别信跨 session 记忆或文档。可用 `client.models.list()` 查。又: - vs _ 一字之差也 400——400 会并排给出合法值和你传的值,逐字符比。

8. 信息在上下文 ≠ 模型会用它

名字已在摘要里,问“我是谁?”却答“无法确认身份”(被当成身份核验);问“我叫什么?”就答“小明”。问法 + system prompt 是否允许,都影响模型愿不愿意用已知信息。

9. 稳定事实走 remember,流程走摘要(L5+L6 合流,本节落点)

名字这种“稳定高价值事实”不该指望有损摘要保住——该走 L5 的 remember 结构化存储。摘要负责压“聊了啥”,remember 负责钉“关于用户的铁事实”。两工具两职责,阶段二 capstone 合起来。

六、Quiz(6 题):Q2/Q3/Q5/Q6 满分;Q1 补讲“单次线性 vs 累计平方”;

Q4 重写 prompt 迭代两版(第一版把 JSON 格式套到所有场景、漏负面约束,已纠正)。

七、埋点

- **阶段二 capstone:**remember(L5)+ 压缩(L6)+ RAG(L7)合成完整客服 agent; Part B 结构化路由、单元测试系统学习,均在此补齐。
- **L7 RAG:**外置策略 + “按相关性检索该取哪几条记忆”。
- **L13 可观测性:**SessionStats(token/调用次数)就是雏形;API 层重试/超时/限流(缺口 4)在此补。